



**Reverse-Tethered Local LLM Inference:
A Systems Feasibility Study Using an Android Flagship as
an External Compute Node for a Legacy Laptop**

Final Dissertation

**TU858
Individual Project**

**Jiuk Kim
A00046494**

Eoin Rogers

School of Computer Science
Technological University, Dublin

1st May 2026

Abstract

Open-weight Large Language Models (LLMs) make local inference attractive for privacy, offline use, and reduced reliance on cloud services. However, transformer inference depends on matrix operations, attention, autoregressive token generation, memory movement, and key-value cache access. These costs are poorly matched to older CPU-only laptops, where even a small local model can produce high first-token latency and modest decode throughput. Modern flagship phones create a useful hardware asymmetry: the laptop may remain the user-facing machine while the phone acts as a nearby external compute node.

This dissertation presents a measurement-driven feasibility study of reverse-tethered local LLM inference. The implemented system keeps a Lenovo Yoga Slim 7-14ARE05 as the client, controller, logger, and practical CPU-only baseline. A Samsung Galaxy S25 Ultra is used as a headless local inference node over USB/Android Debug Bridge (ADB) forwarding. Three conditions are compared: Yoga CPU local inference, S25 CPU inference, and S25 OpenCL/GPU inference.

The evaluation uses Llama 3.2 1B Instruct in GGUF Q4_0 quantization, deterministic generation settings, and a fixed synthetic prompt-fixture suite grouped by prompt-token length. The laptop-side harness sends prompts, consumes streamed responses, records laptop-boundary Time-to-First-Token (TTFT), and computes decode-phase tokens per second (decode TPS). Semantic answer quality is outside scope because the study evaluates systems behavior rather than task accuracy.

In warm runs, the S25 OpenCL/GPU condition improved median TTFT over the Yoga CPU baseline by approximately $3.01\times$ for short prompts, $3.88\times$ for medium prompts, and $3.90\times$ for long prompts. Decode TPS was mixed or broadly comparable rather than universally superior. The study therefore evaluates the implemented CPU and OpenCL/GPU paths, not Neural Processing Unit (NPU) acceleration, and it measures the complete laptop-boundary reverse-tethered system rather than isolating transport from phone compute. The result is a bounded feasibility finding: reverse tethering can materially improve first-token responsiveness under controlled local-inference conditions, but it is not a universal acceleration claim.

Declaration

I hereby declare that the work described in this dissertation is, except where otherwise stated, entirely my own work and has not been submitted as an exercise for a degree at this or any other university.

Signed:



Date:

04/05/26

Acknowledgements

The completion of this dissertation was supported by the guidance of the project supervisor, Eoin Rogers.

Table of Contents

Abstract	1
Declaration	2
Acknowledgements	3
1. Introduction.....	5
2. Background and Related Work	7
3. Theory / Technical Foundations	10
4. System Design and Architecture.....	13
5. Implementation	15
6. Experimental Methodology	17
7. Results.....	21
8. Testing and Evaluation	28
9. Discussion, Project Management, and Future Work	30
Bibliography	34
Appendices.....	36
Appendix A: Prompt-fixtue suite summary.....	36
Appendix B: Condition matrix and regimes	36
Appendix C: Benchmark commands and workflow notes	37
Appendix D: Codebase structure	39
Appendix E: Representative JSONL record categories	40
Appendix F: Result provenance.....	41
Appendix G: Test evidence.....	42
Appendix H: Technical-user evaluation template.....	43

1. Introduction

1.1 Motivation and problem context

Large Language Models (LLMs) are now common in everyday software workflows, but most access still depends on cloud-hosted inference. Cloud services are convenient, yet they require prompts and generated content to leave the user's device. For personal notes, documents, source code, and other private material, local inference is attractive because it keeps computation nearby and can continue without a network connection.

The difficulty is that transformer inference is accelerator- and memory-intensive. A decoder-style LLM repeatedly applies matrix and vector operations, maintains a key-value (KV) cache, and generates tokens one step at a time. These operations can benefit from parallel compute units and high memory bandwidth. On an older central processing unit (CPU)-only laptop without a discrete graphics processing unit (GPU) or neural processing unit (NPU) path, even a small local model can therefore produce high first-token latency and limited generation throughput. Prior measurement studies show that mobile and edge LLM performance is shaped by memory bandwidth, quantization, runtime choice, dynamic voltage and frequency scaling (DVFS), and device state rather than raw arithmetic capability alone [1], [2], [3].

Modern mobile phones create a useful hardware asymmetry. Many older laptops remain adequate as keyboards, displays, development environments, and file-management devices, but lack recent AI acceleration hardware. Flagship smartphones, by contrast, integrate mobile CPUs, GPUs, and NPUs inside a tightly constrained system-on-chip (SoC). This motivates a reverse-tethered design in which the laptop remains the user-facing machine while the phone is used as a nearby external compute node. The comparison with an external GPU is only motivational: the phone is not a transparent PCIe accelerator, but a local inference server reached through a controlled USB transport path using Android Debug Bridge (ADB).

1.2 Research question and scope

This dissertation asks: "Can a modern Android flagship, when used as a reverse-tethered external compute node, provide measurable local LLM inference acceleration over a legacy CPU-only laptop under fixed model, quantization, and generation settings?"

The question is operationalized using laptop-boundary Time-to-First-Token (TTFT), decode-phase tokens per second (decode TPS), and warm, cold, and soak operating regimes. The implemented comparison uses a practical laptop CPU-only baseline, a phone CPU condition, and a phone Open Computing Language (OpenCL)/GPU condition. NPU acceleration was considered as a possible future acceleration path because mobile SoCs increasingly include AI-specific processors, but the final implementation evaluates CPU and OpenCL/GPU paths only. The measurement boundary is the laptop client: for phone conditions, transport through

ADB/USB is deliberately included in TTFT because it is part of the system experienced by the laptop user.

1.3 Contributions and final outcome

The work contributes a proof-of-concept reverse-tethered architecture, a benchmark framework with structured logging and aggregation, a fixed synthetic prompt-fixture methodology, and an empirical comparison across warm, cold, and soak regimes. The final evidence shows that the S25 Ultra OpenCL/GPU condition materially improves warm laptop-boundary TTFT compared with the Yoga CPU-only baseline. Median TTFT speedups are approximately $3.01\times$ for short prompts, $3.88\times$ for medium prompts, and $3.90\times$ for long prompts. Decode TPS is mixed or broadly comparable, so the supported claim is phase-specific: the system improves first-token responsiveness under controlled local-inference conditions, not every aspect of inference performance.

The scope is deliberately bounded. The study does not evaluate semantic answer quality, does not isolate phone compute from transport, and does not claim causal thermal throttling from soak data. These limits make the final claim narrower, but more defensible.

1.4 Dissertation structure

Chapter 2 reviews related work on on-device LLM inference, mobile benchmarking, phase-aware serving, heterogeneous mobile compute, Android Debug Bridge, and llama.cpp. Chapter 3 explains the technical foundations used by the evaluation, including prefill, decode, quantization, execution paths, and measurement metrics. Chapter 4 describes the implemented reverse-tethered architecture, while Chapter 5 summarizes the benchmark implementation. Chapter 6 presents experimental methodology and Chapter 7 reports the empirical results. Chapter 8 describes testing and evaluation. Chapter 9 discusses the findings, threats to validity, project-management outcome, future work, and final conclusion. Technical provenance, command details, and extended records are placed in the appendices so that the main body remains a continuous academic report rather than a repository inventory.

2. Background and Related Work

2.1 On-device and mobile LLM inference

Local LLM inference is motivated by privacy, availability, and reduced cloud dependency. The mobile measurement studies **MELTing Point** and **Large Language Models on Mobile Devices** show that small and quantized LLMs can run on consumer devices, but that performance varies across devices, frameworks, model sizes, prompt lengths, and execution engines [2], [3]. These studies are important because they caution against reasoning from hardware specifications alone. A phone with a modern SoC may be promising, but only measurement can show whether the complete software path is actually useful.

On-device inference is also constrained by memory, energy, and runtime behavior. Quantization reduces the model footprint and can make execution feasible, but it does not remove memory-bandwidth or device-state bottlenecks [2], [3]. Mobile benchmarking work further shows that CPU threading, DVFS, specialized instructions, GPU utilization, and inference-engine choice can substantially affect performance [4]. For that reason, the S25 Ultra OpenCL/GPU result is treated as an empirical condition in a specific software stack, not as a general hardware theorem.

2.2 Phase-aware LLM serving and user-perceived latency

The distinction between prompt processing and token generation is central to the project. **DistServe** describes TTFT as the latency associated with prefill and TPOT as the time per output token during decoding [5]. **Splitwise** similarly characterizes prompt computation as more compute-intensive and token generation as more memory-intensive [6]. **Andes** adds a user-experience perspective by arguing that text-streaming systems should deliver the first token promptly and subsequent tokens at a smooth, digestible pace [7].

This project does not implement phase-disaggregated serving. Instead, it borrows the metric distinction. The reverse-tethered system may reduce first-token waiting time without producing the same improvement in decode throughput. Separating TTFT from decode TPS is therefore necessary to avoid a misleading single speed claim.

2.3 Heterogeneous mobile compute and NPU difficulty

Modern mobile SoCs include multiple compute units. Qualcomm describes heterogeneous mobile computing as a combination of CPU, GPU, NPU, memory subsystem, and software stack, each suited to different workloads [8]. As a vendor whitepaper, this is useful for explaining the design motivation, but not for proving this project’s performance.

Mobile GPU capability is useful to an inference workload only when the runtime exposes an appropriate backend and maps model operations onto that backend. Mobile-AI benchmarking work treats observed performance as the product of hardware, software stack, framework, numeric format, and code path rather than hardware capability alone [20]. In this project,

OpenCL served as the available llama.cpp-supported heterogeneous-compute interface for the phone GPU path [19], [22]. CUDA, the more popular and well-known interface solution from NVIDIA, was not applicable because the target phone used a Qualcomm Adreno GPU rather than an NVIDIA GPU. The implemented acceleration path is therefore described as OpenCL/GPU rather than CUDA [23].

Academic NPU systems show both potential and difficulty. **llm.npu** demonstrates that mobile NPUs can reduce prefill latency, but also explains the engineering barriers: variable-length prompts conflict with static-shape NPU execution, LLM quantization may not match supported NPU matrix operations, and floating-point operations such as LayerNorm and Attention remain difficult to remove [9]. **PowerInfer-2** also shows that smartphone LLM acceleration often requires custom scheduling and I/O-computation pipelining rather than simply enabling an accelerator flag [10]. For that reason, this dissertation discusses NPU as related work and future work only; the implemented acceleration path is OpenCL/GPU.

2.4 Offloading, collaboration, and related positioning

Reverse tethering is related to computation placement, but differs from cloud offloading and distributed inference. **Neurosurgeon** and **Edgent** partition DNN inference between mobile and cloud or edge servers to reduce latency and energy [11], [12]. **LinguaLinked** and **EdgeShard** explore distributed or sharded LLM inference across multiple devices [13], [14]. These systems show that computation placement matters, but they also introduce partitioning, communication, and scheduling complexity that this dissertation avoids.

This project runs the full model on one endpoint per condition: the laptop CPU, the phone CPU, or the phone OpenCL/GPU path. That simpler design makes the comparison easier to interpret. Device-side optimization systems such as **EdgeLLM**, **LLMCad**, **Ghidrah**, and **LLM in a Flash** demonstrate further possibilities, including speculative decoding and memory-aware loading [15]–[18], but those techniques are future extensions rather than implemented parts of this work.

2.5 Project runtime technologies: ADB and llama.cpp

Android Debug Bridge (ADB) is an official Android developer command-line tool for communicating with Android devices. In this project, ADB provides device control and a local TCP forwarding mechanism over USB, allowing the laptop client to reach a server bound to the phone’s localhost interface. Its main strength is that it creates a controlled local transport path without using cloud networking or public Wi-Fi. Its limitation is also methodological: the benchmark measures the complete laptop-boundary path, so ADB overhead is included in phone-condition TTFT but is not decomposed from phone-side tokenization, prefill, decode, or server streaming overhead [24].

llama.cpp is an open-source local LLM inference runtime and server implemented in C/C++. Its server provides LLM REST APIs, OpenAI-compatible chat-completion routes, and support for

CPU and GPU execution of quantized models. llama.cpp also requires models to be available in GGUF format, which made it suitable for using the same Llama 3.2 1B Instruct Q4_0 GGUF model across laptop and phone conditions [25], [26]. The use of llama.cpp provides a comparable server interface across conditions, but it does not guarantee identical backend behavior across Windows CPU, Android CPU, and Android OpenCL/GPU builds.

3. Theory / Technical Foundations

3.1 Autoregressive transformer inference

Generative LLMs process text as tokens. A decoder-style model predicts the next token from the current context, appends that token, and repeats the process until a stopping condition is reached. This autoregressive structure makes inference sequential after the prompt has been processed. A classifier may process input once, but a generative model may require hundreds of dependent forward passes for one response.

The sequential nature of generation explains why latency and throughput must be separated. A system can feel responsive if the first token appears quickly even when the rest of the answer streams at a similar pace to another system. Conversely, high sustained token throughput can still feel poor if the first token is delayed.

3.2 Prefill, decode, and the KV cache

LLM inference is commonly divided into prefill and decode. During prefill, the model processes the prompt and prepares the attention state needed to generate the first token. During decode, it generates subsequent tokens one at a time while reusing cached key and value vectors. The KV cache avoids recomputing previous context, but it increases memory pressure and requires repeated access during generation.

Longer prompts primarily stress prefill and TTFT. Decode has limited token-level parallelism and can become memory-bandwidth constrained. This is why the project expected the S25 Ultra OpenCL/GPU condition to show stronger gains in TTFT than in decode TPS.

Table 3.1: Inference-phase comparison

Phase	Main work	Typical bottleneck	Project metric
Prefill	Process full input prompt and prepare first token	Prompt-length-dependent compute and memory movement	Laptop-boundary TTFT
First-token delivery	Return first generated content token to the laptop	Server, transport, and first-token compute	TTFT
Decode	Generate subsequent tokens autoregressively	Sequential dependency and memory bandwidth	Decode TPS
Full response	Complete streamed request	Combination of TTFT, output length, and decode speed	Total latency as supporting context

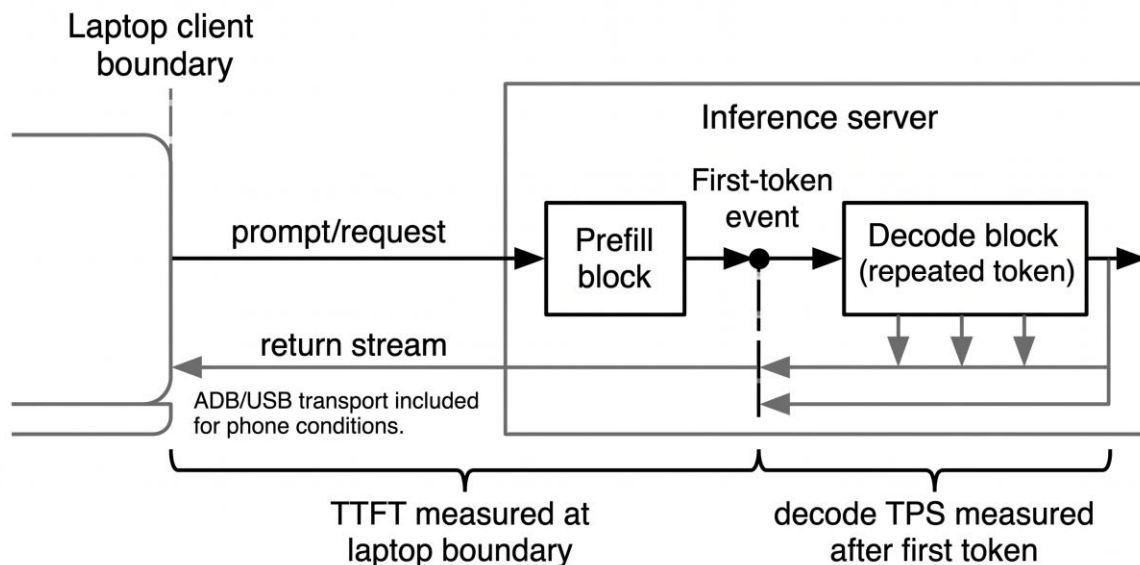


Figure 3.1: Prefill/decode and measurement-boundary diagram showing TTFT at the laptop boundary and decode TPS after the first token.

3.3 Quantization and workload feasibility

Quantization reduces the memory footprint of model weights and is central to local and mobile LLM deployment [2], [3]. This study uses a 1B-class GGUF model with Q4_0 quantization. GGUF is the model file format used by llama.cpp, and Q4_0 is the fixed low-bit quantization variant selected for compatibility across the Yoga CPU, S25 CPU, and validated S25 OpenCL/GPU paths [26]. The project’s OpenCL validation artifacts also require Q4_0 for the implemented Adreno OpenCL path, making it a controlled like-for-like choice for the CPU and OpenCL/GPU comparisons. This does not imply that Q4_0 preserves semantic quality; the study measures systems performance rather than answer quality.

3.4 Execution paths and metrics

CPU execution is flexible and forms the practical baseline for the Lenovo Yoga. OpenCL/GPU execution is the implemented phone acceleration path because the S25 Ultra exposes an Adreno GPU path that could be built and exercised through the project’s OpenCL-enabled llama.cpp workflow. NPU execution was not used because it would have required a separate vendor-SDK or graph-runtime integration, static-shape handling, operator and quantization compatibility work, and validation beyond the time available for the core benchmark [8], [9]. CUDA was not a candidate for the phone path because CUDA targets NVIDIA GPUs, while the S25 Ultra uses an Adreno GPU. The implemented path is therefore described as OpenCL/GPU, not CUDA or NPU.

The benchmark metrics are operational laptop-side measurements, not universal mathematical definitions of LLM performance. They are consistent with the TTFT/TPOT and prefill/decode distinction used in LLM serving literature [5]–[7], but the exact formulas below are this project’s measurement definitions at the laptop-boundary point.

TTFT is the elapsed time from the benchmark harness sending the request on the laptop to the first non-empty streamed content token received by the laptop harness:

$$TTFT_{ms} = (t_{first_token} - t_{request_sent}) \times 1000 \quad (3.1)$$

For phone conditions, this boundary intentionally includes ADB/USB forwarding, phone-side server handling, tokenization/prompt processing, prefill, first-token generation, and first-token delivery back to the laptop. This makes TTFT a user-facing reverse-tethered system metric rather than an isolated phone-compute metric.

Decode TPS is computed after the first token has arrived:

$$Decode\ TPS = \frac{N_{gen} - 1}{t_{final_token} - t_{first_token}} \quad (3.2)$$

Decode TPS is only reported when *generated_token_count* > 1 and $t_{final_token} > t_{first_token}$.

It therefore measures post-first-token decode-phase throughput rather than total request speed. The first generated token is excluded from the decode numerator because its wait time is already represented by TTFT.

The aggregation reports p50 TTFT, p95 TTFT, and mean decode TPS over successful final-evidence records after filtering. p50 is the median, p95 is the 95th percentile used as a tail-latency indicator, and mean decode TPS is the arithmetic average of successful decode-throughput values in a group. Failed final records are retained for failure counts and error-rate reporting, but they are excluded from TTFT and decode-TPS arithmetic where appropriate. Appendix E shows the raw JSONL fields used for these measurements, and Appendix F identifies the final result-provenance directories.

4. System Design and Architecture

4.1 Design rationale

The system is designed as a benchmarkable research artifact. The laptop remains the user-facing and measurement-facing machine, while the phone acts as a local external compute node. The design keeps inference cloud-independent, compares execution paths under fixed conditions, and records the metrics required to evaluate the research question.

The key architectural decision is that the complete path is measured. The phone is not treated as an isolated processor. In phone conditions, the laptop sends the request through ADB/USB to the phone server and receives streamed tokens back through the same path. This reflects what a laptop user would experience.

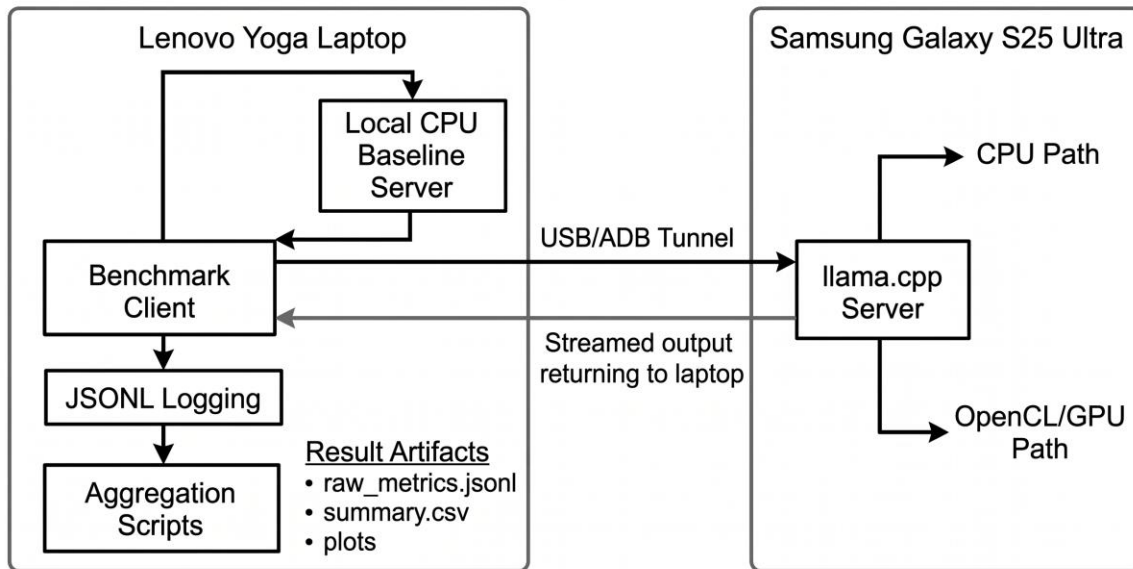


Figure 4.1: Implemented reverse-tethered architecture, including local laptop baseline, ADB/USB path, phone CPU/OpenCL/GPU paths, raw logging, and aggregation.

4.2 Components and runtime flow

The laptop client controls benchmark execution, loads prompts, sends streaming requests, records first-token and final-token timing, and writes JSONL records. The local baseline path targets a laptop-side llama.cpp server running as a practical CPU-only baseline. The phone path uses ADB forwarding to expose a phone-side llama.cpp server to the laptop. The same laptop-side measurement logic is used for all conditions.

The OpenCL/GPU condition used an OpenCL-enabled llama.cpp server configuration consistent with the llama.cpp OpenCL backend documentation [19] and was validated through the project’s OpenCL/GPU build evidence. The dissertation treats this as the implemented phone OpenCL/GPU condition. This wording does not assume that every layer was fully offloaded to

the GPU solely because a high `-ngl` value was requested; exact backend behavior, offload logs, and build details remain documented reproducibility variables. The implementation therefore compares laptop CPU, phone CPU, and phone OpenCL/GPU paths, but it does not claim NPU acceleration or complete GPU offload beyond the available validation evidence.

A benchmark run begins with condition and prompt selection. The client checks readiness where applicable, sends a streaming request, records the request-sent time, identifies the first non-empty content token, records subsequent streamed tokens, writes the raw record, and later aggregates accepted records. Failed records are retained for error-rate analysis rather than silently discarded.

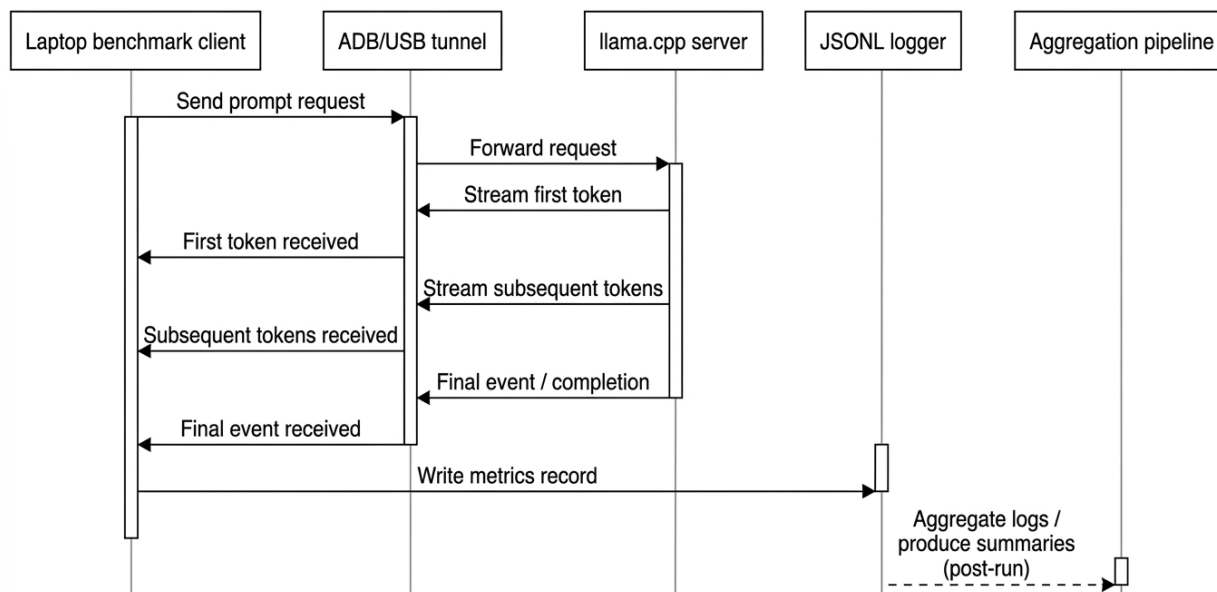


Figure 4.2: Benchmark request and logging sequence showing request forwarding, streamed token return, laptop-side metric recording, JSONL logging, and post-run aggregation.

4.3 Scope boundaries

The architecture deliberately excludes cloud inference, public API inference, model partitioning, multi-phone execution, NPU acceleration, semantic scoring, and production chat UX. These exclusions narrow the artifact, but they also make the results easier to interpret: each benchmark condition corresponds to one complete inference endpoint.

5. Implementation

5.1 Codebase organization

The implementation is a research benchmark framework rather than a polished consumer application. Its codebase separates benchmark control, configuration files, server workflow documentation, scripts, analysis, tests, and results. This separation keeps the empirical workflow reproducible and avoids hiding the experiment inside ad hoc commands. Appendix D expands Table 5.1 into a practical repository map, showing where to locate the benchmark client, configuration files, runbook documentation, aggregation scripts, automated tests, and final result artifacts.

At the user-facing level, the benchmark is driven through a command-line interface (CLI), which is appropriate for a research artifact because it makes prompt selection, condition selection, regime selection, and output directories explicit.

Table 5.1: Repository component summary

Area	Role	Examples
client	Laptop-side CLI, matrix runner, streaming benchmark, metrics, and preflight checks	cli.py, benchmark.py, matrix.py, metrics.py, preflight.py
configs	Prompt fixtures, condition matrix, and server metadata schema	dataset_suite_v1.json, matrix.json
scripts	ADB setup, phone server launch, metadata capture, prompt-token checks	adb_bootstrap.sh, launch_phone_server.sh
analysis	Final-evidence filtering, aggregation, plots, and report assets	aggregate.py, export_paper_assets.py
docs	Runbooks and reproducibility notes used to build the user manual	logging_schema.md, phone_server_runbook.md
tests	Automated validation of metrics, streaming, CLI, preflight, aggregation, and mock end-to-end behavior	test_metrics.py, test_benchmark.py, test_aggregate.py

5.2 Benchmark client and streaming measurement

The laptop client builds completion requests, targets either the local endpoint or the ADB-forwarded phone endpoint, and consumes server-sent streaming responses. Timing is captured on the laptop. The first-token timestamp is set only when a non-empty streamed content token arrives, which prevents metadata-only chunks from being mistaken for user-visible output. The final-token timestamp and server-reported token count support decode TPS computation after the first token.

The implementation writes one structured JSON object per run to a JSONL file, `raw_metrics.jsonl`. Each record includes condition, regime, prompt metadata, timing fields, computed metrics, cache-policy metadata, and error information where relevant. This makes the result pipeline auditable from raw evidence to aggregated tables. Appendix E includes a condensed example of the JSONL record structure used by the benchmark logs.

5.3 Prompt fixtures, conditions, and server workflow

The final workload is encoded as a controlled synthetic prompt-fixture suite with short, medium, long, and soak tiers, while smoke and development prompts are kept separate from final evidence. The condition matrix defines cold, warm, and soak regimes. The phone workflow documents Termux setup, ADB forwarding, CPU mode, and OpenCL/GPU mode, supporting comparison between laptop CPU, phone CPU, and phone OpenCL/GPU paths.

5.4 Logging, aggregation, and implementation limits

The final summaries used by the dissertation are stored in two authoritative aggregation directories: `results/final_summaries_all_regimes/aggregation_20260426_232956/` and `results/final_summaries_all_regimes_by_prompt/aggregation_20260426_232959/`. These directories contain grouped `summary.csv` and `summary.md` outputs, by-prompt summaries, plots, and raw export artifacts. The Results chapter interprets these artifacts, while Chapter 5 only establishes that the implementation provides a reproducible pipeline from raw JSONL records to report-ready outputs.

The final-evidence filter also requires non-empty `source_article_id` provenance. In this project, that field is used as source/offline fixture provenance for the controlled synthetic prompt suite; it should not be read as evidence that the evaluation used a public article dataset. This provenance field helps prevent malformed or non-final prompt records from entering the dissertation summaries.

The main implementation challenges were managed through scope control. OpenCL/GPU became the implemented phone acceleration condition; NPU acceleration, causal thermal analysis, isolated ADB decomposition, and a tuned CPU baseline were treated as limitations or future work. These decisions narrowed the artifact but improved the integrity of the empirical claim.

6. Experimental Methodology

6.1 Research design

The empirical study evaluates whether the reverse-tethered phone path improves user-perceived inference performance over a practical CPU-only laptop baseline. The primary expectation was that the S25 Ultra OpenCL/GPU condition would reduce laptop-boundary TTFT, especially for medium and long prompts. Decode TPS was not expected to improve universally because autoregressive decode has limited sequential parallelism and can be memory-bandwidth constrained. The phone CPU condition was included as a diagnostic comparison. Soak runs were included to observe sustained trends, not to prove thermal causality.

Table 6.1: Hardware used in empirical study

Device	Role	Processor / accelerator	Memory / OS
Lenovo Yoga Slim 7-14ARE05	Client, controller, logger, local baseline	AMD Ryzen 7 4700U CPU; no discrete GPU/NPU used	16 GB RAM; Windows 11 Education
Samsung Galaxy S25 Ultra SM-S938N	Reverse-tethered phone inference node	Snapdragon 8 Elite; Adreno 830; NPU not implemented	12 GB RAM; Android 15

The USB/ADB link is treated as transport rather than hardware in Table 6.1. For phone conditions, that transport is part of the measured laptop-boundary system and is described in the benchmark-condition table and measurement-boundary discussion.

Table 6.2: Model and generation settings

Setting	Value	Purpose
Model	Llama 3.2 1B Instruct Q4_0 GGUF	Fixed workload across all compared paths
Quantization	Q4_0	Fixed low-bit format used across laptop CPU, phone CPU, and validated phone OpenCL/GPU conditions
Context length	2048	Controls prompt/context state
Max new tokens	512	Provides decode window while bounding output
Temperature / seed	0.0 / 42	Reduces sampling variation
Cache policy	Disabled or cleared by restart	Prevents hidden prompt-cache reuse in final evidence

6.2 Prompt fixtures and benchmark conditions

The final prompts are a fixed synthetic prompt-fixture suite grouped by prompt-token length, summarized in Appendix A. This is methodologically appropriate because the dependent variables are systems metrics rather than answer quality: TTFT, decode TPS, latency behavior, stability, and reproducibility. Prompt-token length is directly relevant to prefill and first-token delay, while holding prompt text constant across conditions isolates execution-path differences. Public semantic datasets such as CNN/DailyMail would introduce task-quality and answer-quality confounds unless semantic scoring were also implemented. For this project, a controlled

systems workload is therefore more defensible than a poorly integrated public summarization benchmark.

Table 6.3: Prompt-fixture tiers

Tier	Prompts	Fixture token range	Regimes
Short	5	107-110	Warm and cold
Medium	5	480-494	Warm and cold
Long	5	1372-1400	Warm and cold
Soak	1	487	Soak only

Table 6.4: Benchmark conditions

Condition ID	Device/path	Purpose	Key limitation
yoga_cpu_local_Q4_0	Yoga local CPU	Practical legacy-laptop baseline	Not a tuned x86 CPU benchmark
s25ultra_cpu_phone_Q4_0	Phone CPU over ADB	Diagnostic reverse-tethered phone baseline	Includes ADB transport and mobile runtime effects
s25ultra_openc1_phone_Q4_0	Phone OpenCL/GPU over ADB	Main implemented phone acceleration path	OpenCL/GPU condition, not NPU or complete offload proof

6.3 Operating regimes and measurement procedure

Cold, warm, and soak are treated as operating regimes rather than separate metrics. In the cold regime, the measured request is the first request after server launch and readiness checks; it captures restart-adjacent first-use effects at the laptop boundary, but it does not include model-load time as a separate elapsed metric. In the warm regime, the server and model are already available and the benchmark measures repeated interactive use. In the soak regime, a fixed prompt is repeated over a sustained run to observe whether TTFT or decode TPS trends change over time. Since reliable temperature telemetry was not integrated, soak results are interpreted as sustained-performance trends rather than causal proof of thermal throttling.

Table 6.5: Operating regimes and final sample counts

Regime	Purpose	Prompt tiers	Final records	Limitation
Warm	Normal repeated operation after server/model availability	Short, medium, long	225	Does not include server launch/model-load time
Cold	First measured request after fresh server launch/readiness	Short, medium, long	45	Subset; duplicated bundled cold folders not counted as extra evidence
Soak	Repeated fixed workload for sustained-performance trends	Soak	205	Not causal thermal measurement

Each run selects a condition, regime, and prompt; prepares the relevant endpoint; performs readiness checks where applicable; sends a streaming request from the laptop; records request-sent, first-token, and final-token timestamps; writes a JSONL record; and then aggregates final-evidence records. Appendix B summarizes the condition matrix, Appendix C lists representative workflow commands, and Appendix E describes the main JSONL record categories.

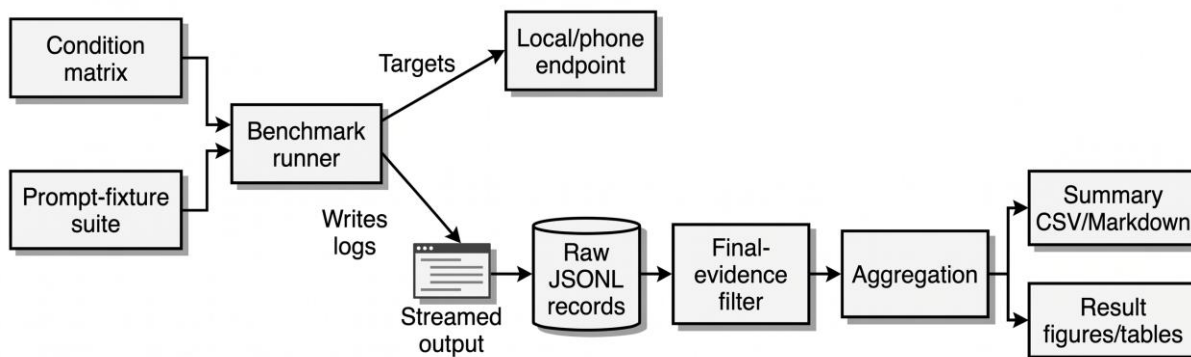


Figure 6.1: Experimental pipeline from condition matrix and prompt fixtures to benchmark execution, raw records, final-evidence filtering, and summaries.

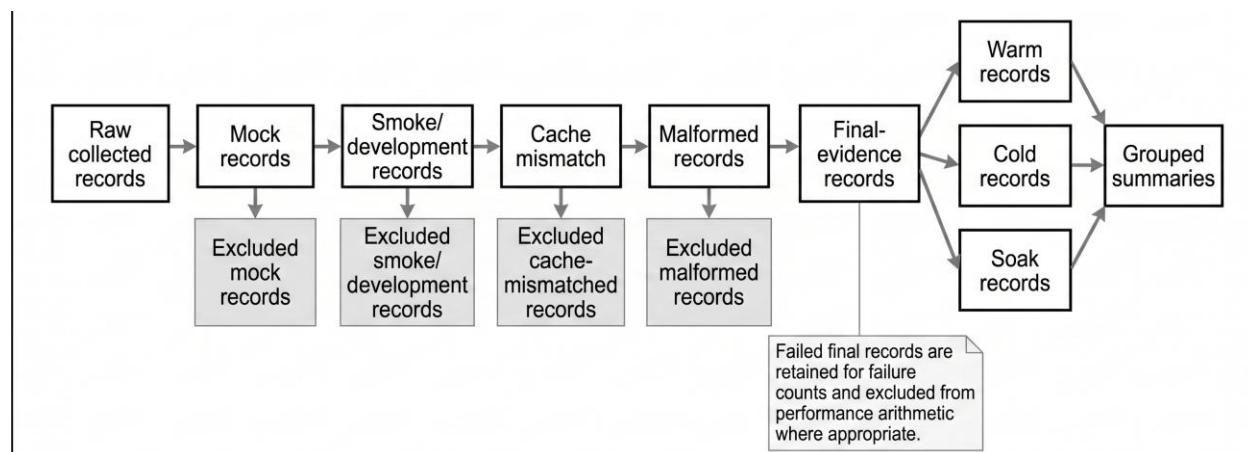


Figure 6.2: Final-evidence filtering: raw records are filtered to exclude mock, smoke/development, cache-mismatched, and malformed records before final evidence is grouped into warm, cold, and soak summaries. Failed final records are retained for failure counts and excluded from performance arithmetic where appropriate.

6.4 Aggregation, fairness controls, and limits

Table 6.6: Final-evidence filtering policy

Policy item	Reason	Effect
Synthetic final suite required	Separates final workload from smoke/development prompts	Only controlled fixtures enter results
Cache disabled or cleared by restart	Avoids hidden prompt-cache reuse	Protects TTFT validity
Mock and malformed records excluded	Prevents development evidence contamination	Keeps final summaries credible
Failures retained in counts	Avoids survivorship bias	Failure/error rate remains visible
Grouped by condition, regime, tier	Supports fair comparison	Enables warm/cold/soak interpretation

The study reports descriptive statistics rather than formal inferential tests. p50 is the median value: half of the successful measured requests in a group are at or below it. p95 is the 95th percentile and is used as a tail-latency indicator. Mean decode TPS is the arithmetic average of successful decode-phase throughput values in a group. Failed final records are retained for failure-rate analysis and excluded from performance arithmetic where appropriate. Appendix F gives the result provenance and final summary directories. These descriptive statistics support a controlled feasibility comparison, not population-level generalization to all devices, models, or software stacks.

7. Results

7.1 Reading the results

This chapter reports the final benchmark evidence. The grouped all-regimes summary is the primary source for condition, regime, and tier comparisons. The by-prompt summary is secondary and used only for variance inspection. Raw JSONL records provide provenance, but raw directories are not naïvely recounted because the aggregation pipeline filters final evidence and avoids duplicate development or bundled cold records. The results are descriptive and specific to the documented hardware, model, prompt fixtures, and runtime stack.

7.2 Warm full-matrix results

The warm full matrix is the primary evidence because it contains 225 final records: 25 records per prompt tier per condition. Figure 7.1 shows the central result: the S25 Ultra OpenCL/GPU condition produces substantially lower laptop-boundary TTFT than the Yoga CPU baseline across all prompt tiers. The difference becomes especially visible for medium and long prompts, where prompt processing dominates more of the first-token wait.

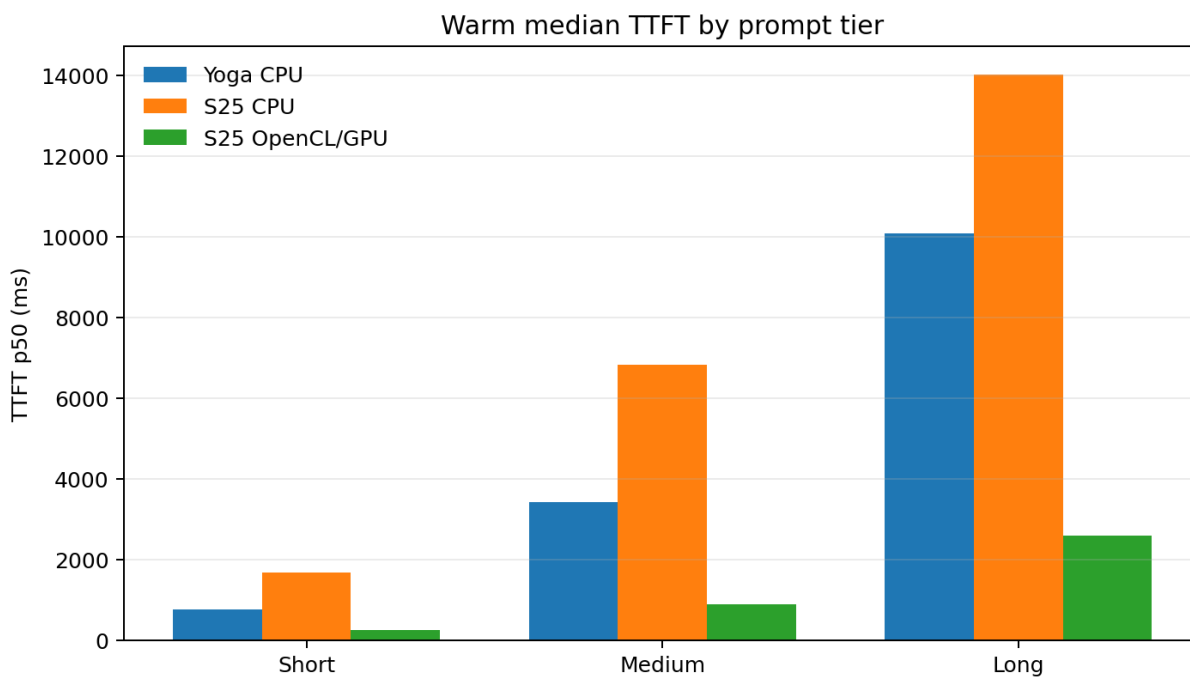


Figure 7.1: Warm TTFT p50 by prompt tier and condition. Lower is better.

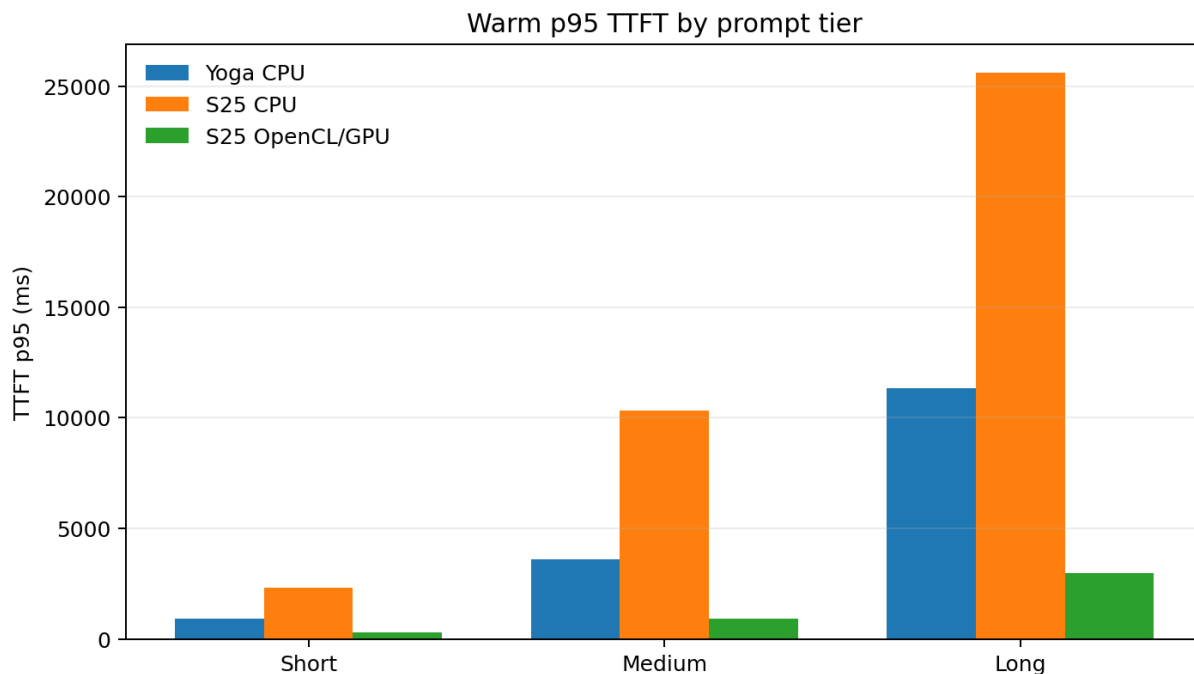


Figure 7.2: Warm TTFT p95 by prompt tier and condition. Lower is better.

Table 7.1: Warm result summary

Condition	Tier	TTFT p50 ms	TTFT p95 ms	Mean decode TPS
Yoga CPU	Short	749.39	909.75	33.96
S25 CPU	Short	1664.09	2330.91	11.70
S25 OpenCL/GPU	Short	248.79	279.01	33.20
Yoga CPU	Medium	3416.14	3600.81	32.34
S25 CPU	Medium	6827.36	10327.06	11.45
S25 OpenCL/GPU	Medium	881.11	900.47	32.94
Yoga CPU	Long	10079.19	11323.99	27.68
S25 CPU	Long	14011.68	25592.99	25.45
S25 OpenCL/GPU	Long	2587.71	2966.12	29.88

7.3 TTFT speedup and decode TPS

The headline TTFT speedup is calculated against the Yoga CPU baseline using median TTFT. The S25 Ultra OpenCL/GPU condition improves median TTFT by approximately $3.01\times$ for short prompts, $3.88\times$ for medium prompts, and $3.90\times$ for long prompts. This is the strongest final result and supports the feasibility of the reverse-tethered OpenCL/GPU path as a first-token responsiveness improvement.

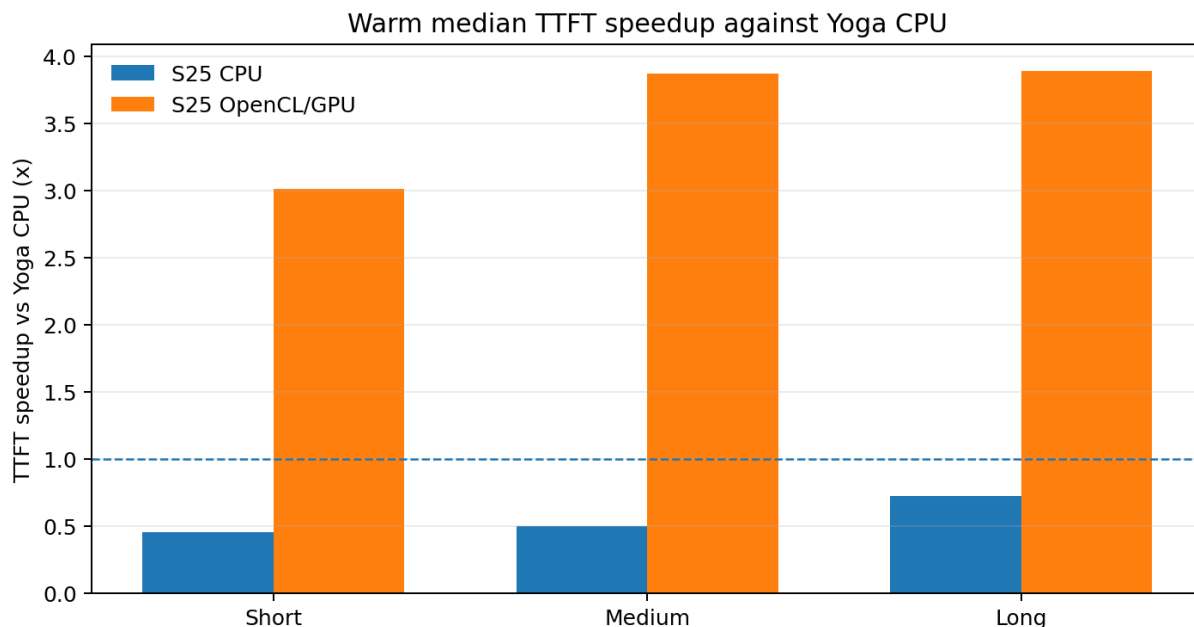


Figure 7.3: Warm median TTFT speedup against the Yoga CPU baseline. Higher is better.

Table 7.2: Warm TTFT speedup versus Yoga CPU baseline

Tier	S25 CPU speedup	S25 OpenCL/GPU speedup	Interpretation
Short	0.45×	3.01×	OpenCL/GPU strongly improves first-token latency; phone CPU is slower.
Medium	0.50×	3.88×	OpenCL/GPU improvement becomes larger as prompt length increases.
Long	0.72×	3.90×	OpenCL/GPU substantially reduces long-prompt first-token waiting time.

Decode TPS is a different story. Figure 7.4 shows that the OpenCL/GPU condition is broadly comparable to the Yoga baseline rather than universally superior. It is slightly below the baseline for short prompts, slightly above for medium prompts, and modestly above for long prompts. This confirms that the contribution is phase-specific: first-token responsiveness improves more clearly than decode-phase throughput.

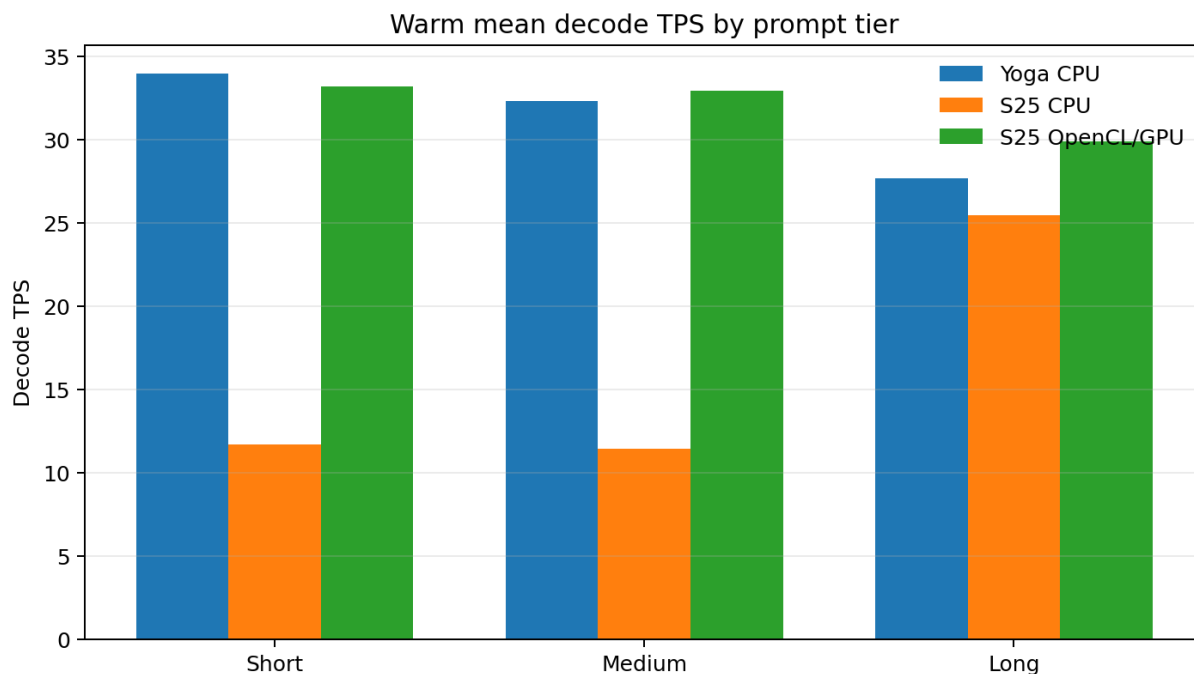


Figure 7.4: Warm mean decode TPS by prompt tier and condition. Higher is better.

Table 7.3: Warm decode TPS ratio versus Yoga CPU baseline

Tier	S25 CPU ratio	S25 OpenCL/GPU ratio	Interpretation
Short	0.34×	0.98×	OpenCL/GPU near parity; phone CPU much slower.
Medium	0.35×	1.02×	OpenCL/GPU effectively comparable to baseline.
Long	0.92×	1.08×	OpenCL/GPU modestly ahead; not a universal throughput win.

7.4 Cold and soak results

The cold subset contains 45 final records and is secondary to the warm full matrix. It nevertheless shows favorable phone TTFT behavior for first measured requests after fresh server launch/readiness. It should not outweigh the warm matrix because it has fewer records and a smaller prompt subset.

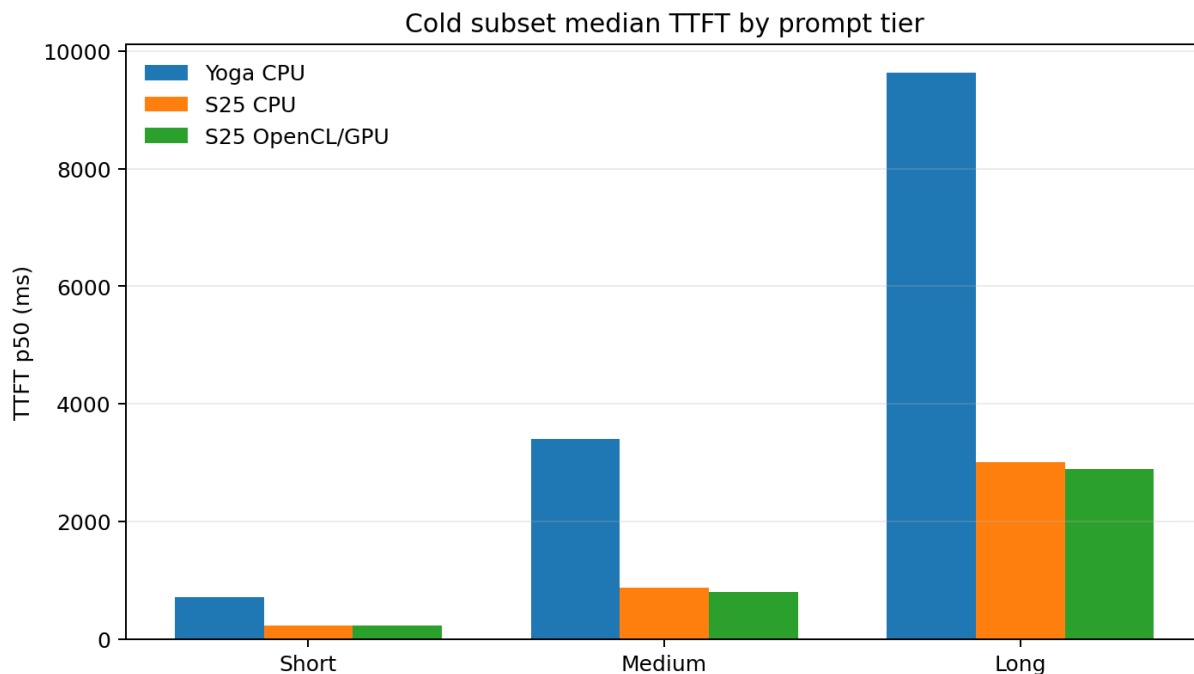


Figure 7.5: Cold subset TTFT p50 by prompt tier and condition. Lower is better.

Table 7.4: Cold subset summary

Condition	Short TTFT ms	Medium TTFT ms	Long TTFT ms	Mean decode TPS range
Yoga CPU	716.98	3396.16	9630.95	25.64-31.79
S25 CPU	220.97	868.93	3013.05	44.21-57.15
S25 OpenCL/GPU	227.19	794.67	2894.18	29.16-36.40

The soak regime contains 205 final records and uses a fixed repeated prompt. The OpenCL/GPU condition retains strong first-token responsiveness in soak, with median TTFT of 901.62 ms compared with 3471.02 ms for the Yoga baseline. Its mean decode TPS is below the Yoga baseline in soak, so the same phase-specific interpretation still applies. Figure 7.6 uses the raw soak record order to show sustained behavior over elapsed time for both TTFT and decode TPS. The raw elapsed-time files confirm that all soak runs exceeded 15 minutes. Since thermal metadata was not reliably captured in the benchmark logs, this plot is used as sustained-performance trend evidence only and not as proof of thermal throttling.

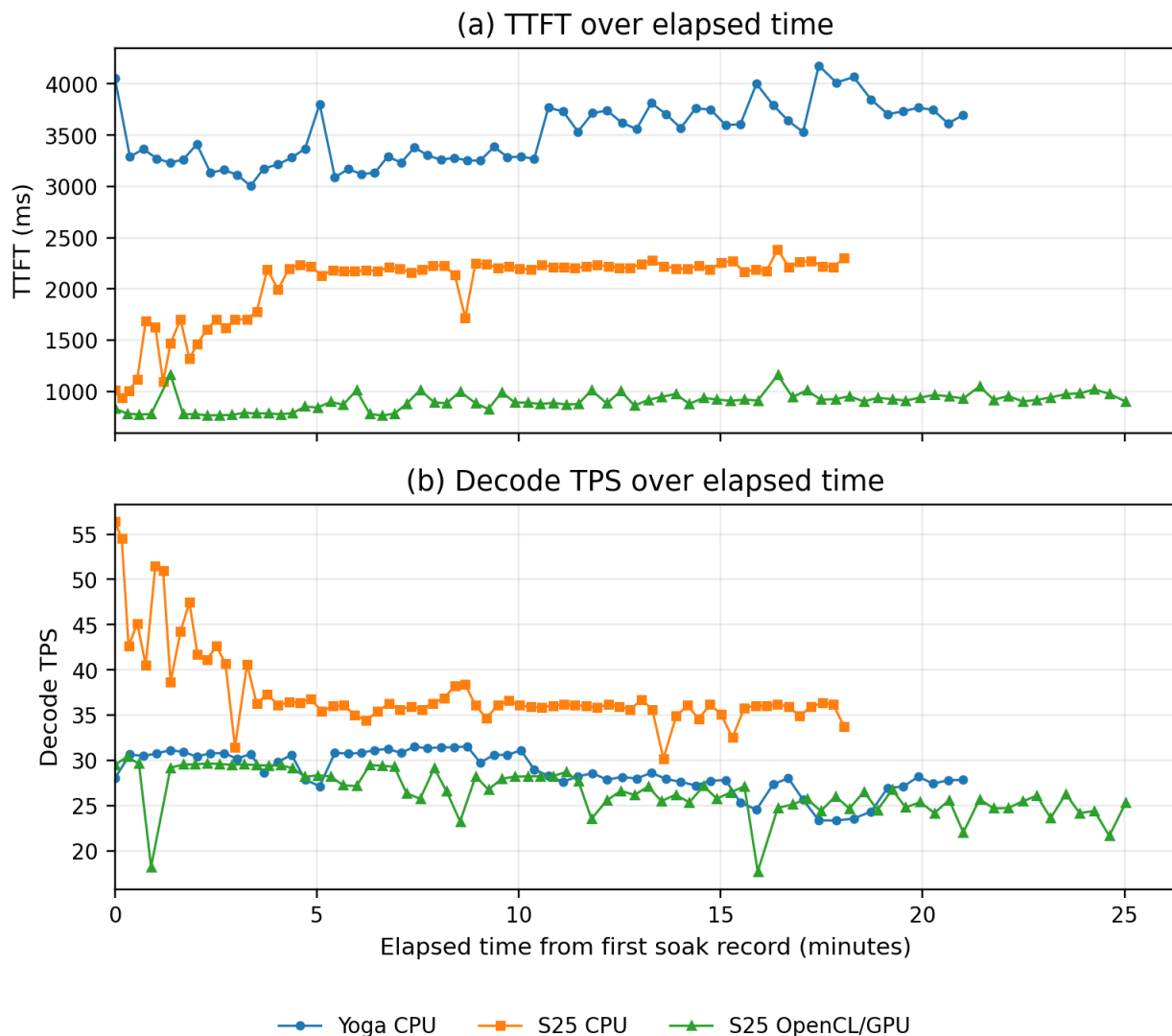


Figure 7.6: Soak performance over elapsed time. Panel (a) shows TTFT and panel (b) shows decode TPS across repeated soak requests. These plots show sustained-performance trends only; thermal metadata was not reliably captured and the figure is not evidence of causal thermal throttling.

Table 7.5: Soak summary

Condition	Final records	TTFT p50 ms	TTFT p95 ms	Mean decode TPS
Yoga CPU	60	3471.02	4015.52	28.79
S25 CPU	70	2194.66	2273.10	37.76
S25 OpenCL/GPU	75	901.62	1014.63	26.60

7.5 Stability and result interpretation

The final grouped evidence reports zero failures across the principal benchmark rows. This supports stability within the controlled final evidence set, but it does not establish production reliability or general robustness outside the measured campaign. The phone CPU condition is useful as a diagnostic comparison but is not the central positive result: it is weak in the warm matrix and more favorable in cold and soak summaries. The safest interpretation is that the OpenCL/GPU condition provides the strongest evidence for improved laptop-boundary TTFT, while decode throughput and phone CPU behavior remain more regime-sensitive.

8. Testing and Evaluation

8.1 Testing versus evaluation

Testing verified that the software framework behaved correctly during development. Evaluation assessed whether the completed system answered the research question. Since this project is a research benchmark framework rather than a consumer chatbot, final evaluation is primarily empirical systems evaluation: the warm, cold, and soak benchmark campaign reported in Chapter 7.

Testing covered the correctness of metric calculation, streaming parsing, first-token detection, prompt/config handling, preflight checks, failure logging, and aggregation filtering. Evaluation covered the finished artifact’s ability to generate controlled evidence, aggregate it, and support a bounded answer to the research question.

8.2 Automated and operational testing

The repository includes automated tests and operational validation hooks for the benchmark framework. These tests cover metric calculation, streaming/Server-Sent Events (SSE) parsing, first-token detection, prompt/config handling, preflight checks, failure logging, and aggregation filtering.

Current unittest discovery evidence showed 152 tests run with 0 failures and 0 errors. The dissertation does not claim that pytest passed. The command `python -m pytest -q` was attempted but did not complete within the tool timeout, while unittest discovery provided the usable current test-run evidence.

Table 8.1: Representative testing and validation evidence

Area	Evidence	Status
Metric calculation	TTFT and decode TPS tests, including invalid decode windows	Passed in unittest discovery
Streaming parser	SSE parsing, first-token detection, final-event handling	Passed in unittest discovery
Prompt/config handling	Prompt-suite integrity and smoke-suite separation	Passed in unittest discovery
Preflight checks	ADB device, forwarding, server health, timeout and refused-connection cases	Passed in unittest discovery
Aggregation	Final-evidence filtering, grouping, p50/p95, decode TPS mean, raw export	Passed in unittest discovery
End-to-end workflow	Final result directories for Yoga, S25 CPU, and S25 OpenCL/GPU	Met through empirical campaign

8.3 Integration and acceptance evaluation

Integration testing focused on the links between components: CLI to prompt configuration, benchmark client to local endpoint, benchmark client to ADB-forwarded phone endpoint, streaming parser to JSONL logging, and JSONL records to aggregation. Some checks are automated with mocks; others are operational because they depend on ADB, Android, and phone server state.

Table 8.2: Acceptance evaluation of the research artifact

Criterion	Evidence	Status
Runs laptop CPU baseline	Yoga warm/cold/soak result directories	Met
Runs phone CPU condition	S25 CPU warm/cold/soak result directories	Met
Runs phone OpenCL/GPU condition	S25 OpenCL/GPU warm/cold/soak result directories	Met
Records laptop-boundary TTFT and decode TPS	Client metrics and raw records	Met
Generates raw JSONL logs and final summaries	Final input and aggregation directories	Met
Excludes non-final evidence	Aggregation filter and tests	Met
Workflow documentation	Runbooks, scripts, configs, and appendices document setup and reproduction	Met as supporting documentation
Production chatbot UX	Not central to final artifact	Not met by design

8.4 Technical-user validation status

A technical-user evaluation template was prepared for a reproducibility and usability review of the benchmark framework, but completed peer feedback was unavailable at the time of writing. The intended evaluation focuses on technical reproducibility rather than consumer usability: whether another technically competent user can understand the project purpose, prepare the environment, validate installation, inspect prompts and conditions, identify the local and phone workflows, run or follow a small benchmark command, locate raw and aggregated outputs, and interpret TTFT and decode TPS. Appendix H contains the prepared evaluation template. No peer outcomes are claimed.

9. Discussion, Project Management, and Future Work

9.1 Answer to the research question

The research question asked whether a modern Android flagship, used as a reverse-tethered external compute node, can provide measurable local LLM inference acceleration over a legacy CPU-only laptop under fixed model, quantization, and generation settings. The answer is positive, but bounded.

Under the fixed Llama 3.2 1B Instruct Q4_0 workload, deterministic generation settings, and controlled synthetic prompt-fixture suite, the S25 Ultra OpenCL/GPU condition substantially improved laptop-boundary TTFT compared with the Yoga CPU-only baseline. In the warm matrix, the median TTFT improvements were approximately $3.01\times$ for short prompts, $3.88\times$ for medium prompts, and $3.90\times$ for long prompts. This is the strongest empirical result of the project.

The result does not show that a smartphone is universally faster than a laptop. Decode TPS was mixed or comparable rather than universally superior. The phone CPU condition was also regime-sensitive: it was weak in the warm matrix but more favorable in the cold and soak summaries. The supported conclusion is therefore phase-specific. The implemented reverse-tethered OpenCL/GPU path improves first-token responsiveness under controlled local-inference conditions, but it does not improve every aspect of inference performance.

Table 9.1: Research question answer by metric

Outcome	Observed result	Interpretation	Limitation
Warm TTFT	OpenCL/GPU speedups of about $3.01\times$, $3.88\times$, and $3.90\times$	Strong evidence for first-token responsiveness improvement	Complete path measured, not isolated phone compute
Warm decode TPS	Near parity or modestly different	Decode throughput not universally improved	One workload and runtime family
Phone CPU	Weak in warm, favorable in cold/soak	Diagnostic condition, not central positive evidence	Cause not isolated
Soak	OpenCL/GPU strong TTFT, mixed decode TPS	Sustained trend evidence	No causal thermal telemetry
Failures	Zero final grouped failures	Stable campaign within scope	Not production reliability

9.2 Interpretation of results and threats to validity

The key interpretation is phase-specific. The S25 OpenCL/GPU condition improves TTFT most clearly for medium and long prompts, while decode TPS remains mixed or broadly comparable. This is not a failed acceleration result: TTFT and decode TPS measure different parts of the interactive generation path. The result therefore supports bounded feasibility for first-token responsiveness, not a general claim that phone execution is faster for every inference phase.

The measurement boundary is intentionally the laptop client. For phone conditions, TTFT includes request handling, ADB/USB forwarding, phone-side server work, prefill, and first-token

delivery back to the laptop. This is appropriate for the reverse-tethered user experience, but it limits attribution. The results show that the complete laptop-boundary phone path is faster for TTFT in the OpenCL/GPU condition; they do not isolate pure phone compute from transport and server overhead.

Several threats to validity remain. First, the hardware scope is narrow: one Lenovo Yoga laptop and one Samsung Galaxy S25 Ultra. The result may not generalize to other laptops, phones, SoCs, GPU drivers, Android versions, or llama.cpp builds. Second, the workload scope is narrow: one 1B-class model, one Q4_0 GGUF file, one context length, deterministic generation settings, and a synthetic prompt-fixture suite. This improves experimental control but limits claims about larger models, different quantization formats, and semantic task quality. Third, the OpenCL/GPU condition is an implemented backend condition, not a proof of complete GPU offload for every operation. Fourth, the final benchmark includes ADB/USB overhead but does not decompose it. Fifth, reliable thermal telemetry was not captured, so soak plots are sustained-performance trend evidence only and are not proof of causal thermal throttling. Sixth, zero failures applies only to the controlled final evidence set; it does not establish production reliability.

Table 9.2: Threats to validity and mitigations

Category	Threat	Mitigation	Remaining limitation
Internal	Timing, streaming parser, or aggregation error	Automated tests, shared client timing, raw export, final-evidence filter	Runtime edge cases may remain
Construct	TTFT and decode TPS do not measure everything	Metrics match research question and phase distinction	No semantic quality, energy, or isolated transport metric
External	Single laptop, phone, model, quantization, runtime	Hardware and workload documented	No broad generalization
Reproducibility	Build, driver, OS, and package differences	Structured records, prompt IDs, documented procedures	Exact environment not perfectly reproducible
Ecological	Controlled prompts and CLI workflow	Appropriate for systems benchmark	Not full consumer UX
Thermal/power	No reliable thermal or energy telemetry	Soak reported only as trend	No causal throttling or energy claim

These limitations narrow the claim but also make it more defensible. The dissertation does not claim NPU acceleration, cloud-equivalent performance, semantic answer quality, production readiness, or general acceleration across all local LLM workloads.

9.3 Project management and deliverable status

The project evolved from a broader reverse-tethered local AI stack concept into a measurement-driven systems feasibility study. The core research question remained stable: the laptop remains the interface while the phone is used as a nearby compute node. The implementation scope became more disciplined as the project progressed. Product-oriented

elements such as a consumer chat interface were treated as non-goals so that the benchmark framework, raw evidence, and result interpretation could be completed reliably.

Several risks were handled through scope control. NPU execution was moved to future work because it would require separate vendor-SDK or graph-runtime integration, static-shape handling, operator compatibility, quantization compatibility, and substantial validation. Thermal causality was removed from the claims because usable thermal telemetry was not captured. ADB overhead decomposition was left as future work because the implemented evaluation measures the complete laptop-boundary system. Public semantic datasets were not used as primary evidence because semantic answer-quality scoring was outside scope; a fixed synthetic prompt-fixture suite was more appropriate for a systems benchmark.

The completed software/research deliverable is a benchmark framework with configuration files, laptop-side measurement logic, ADB/phone workflow documentation, raw JSONL logging, aggregation scripts, plots, result summaries, and automated tests. The final empirical package supports the three benchmark conditions: Yoga CPU, S25 CPU, and S25 OpenCL/GPU. The dissertation is the main written deliverable. Supporting workflow documentation is provided through the existing runbooks, scripts, configuration files, and appendices rather than as central research evidence.

The project is complete as a feasibility artifact, not as a commercial local-AI product. Completeness here means that the system can run the benchmark conditions, collect raw evidence, aggregate results, and support a bounded answer to the research question.

9.4 Future work

The highest-priority future work is an NPU/vendor-SDK path. The related literature shows that NPUs can be promising for prefill, but also that practical integration depends on graph preparation, static-shape handling, operator coverage, quantization format, and scheduling [8]–[10]. A future implementation should treat NPU as a distinct backend rather than as a simple replacement for OpenCL/GPU.

The second priority is ADB/USB overhead decomposition. The current result correctly measures what the laptop user experiences, but it cannot separate transport, server overhead, prompt tokenization, prefill, and first-token generation. A controlled microbenchmark would make the attribution clearer, especially for short prompts where transport and server overhead may be more visible.

The third priority is a tuned laptop CPU baseline. The current Yoga CPU condition is a practical legacy-laptop baseline, not necessarily the strongest possible x86 CPU configuration. A future study should compare the reverse-tethered phone path against a rebuilt and tuned local CPU runtime so that practical benefit and best-case CPU benefit can be separated.

Further work should add reliable thermal, frequency, and power telemetry; broaden the hardware matrix across more laptops and phones; test larger models and alternative quantization formats;

and add a public semantic benchmark as a secondary evaluation once semantic scoring is included. Adaptive routing is also a natural extension: the laptop could choose local CPU or phone OpenCL/GPU execution depending on prompt length, device readiness, and sustained-performance state. More advanced systems ideas such as speculative decoding, model collaboration, or multi-device inference should be treated as later extensions rather than as changes to the core result.

9.5 Final conclusion

This dissertation demonstrates a bounded but meaningful feasibility result. A Samsung Galaxy S25 Ultra used as a reverse-tethered OpenCL/GPU inference node can materially reduce laptop-boundary Time-to-First-Token for a fixed Llama 3.2 1B Instruct Q4_0 workload compared with a Lenovo Yoga CPU-only baseline. The effect is strongest for first-token responsiveness and becomes more pronounced as prompt length increases.

The same evidence does not support a universal acceleration claim. Decode TPS is mixed or comparable, the measurement includes ADB/USB transport, NPU acceleration was not implemented, and the hardware/model scope is narrow. The main contribution is therefore not a production-ready local AI system, but a reproducible benchmark artifact and empirical result showing that reverse-tethered phone inference can be useful under controlled local-inference conditions.

Bibliography

- [1] A. Vaswani et al., “Attention Is All You Need,” in Advances in Neural Information Processing Systems, 2017.
- [2] S. Laskaridis, K. Katevas, L. Minto, and H. Haddadi, “MELTING Point: Mobile Evaluation of Language Transformers,” in Proceedings of ACM MobiCom, 2024.
- [3] X. Li, Z. Lu, D. Cai, X. Ma, and M. Xu, “Large Language Models on Mobile Devices: Measurements, Analysis, and Insights,” in Proceedings of EdgeFM/MobiSys, 2024.
- [4] J. Xiao, Q. Huang, X. Chen, and C. Tian, “Large Language Model Performance Benchmarking on Mobile Platforms: A Thorough Evaluation,” 2024.
- [5] Y. Zhong et al., “DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving,” in Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation, 2024.
- [6] P. Patel et al., “Splitwise: Efficient Generative LLM Inference Using Phase Splitting,” 2024.
- [7] J. Liu et al., “Andes: Defining and Enhancing Quality-of-Experience in LLM-Based Text Streaming Services,” 2024.
- [8] Qualcomm Technologies, Inc., “Unlocking on-device generative AI with an NPU and heterogeneous computing,” Whitepaper, 2024.
- [9] D. Xu et al., “Fast On-device LLM Inference with NPUs,” in Proceedings of ASPLOS, 2025.
- [10] Z. Xue et al., “PowerInfer-2: Fast Large Language Model Inference on a Smartphone,” 2024.
- [11] E. Li, Z. Zhou, and X. Chen, “Edge Intelligence: On-Demand Deep Learning Model Co-Inference with Device-Edge Synergy,” 2018.
- [12] Y. Kang et al., “Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge,” in Proceedings of ASPLOS, 2017.
- [13] J. Zhao et al., “LinguaLinked: Distributed Large Language Model Inference on Mobile Devices,” in Proceedings of ACL System Demonstrations, 2024.
- [14] M. Zhang, J. Cao, X. Shen, and Z. Cui, “EdgeShard: Efficient LLM Inference via Collaborative Edge Computing,” 2024.
- [15] D. Xu et al., “EdgeLLM: Fast On-Device LLM Inference With Speculative Decoding,” IEEE Transactions on Mobile Computing, vol. 24, no. 4, 2025.
- [16] D. Xu et al., “LLMCad: Fast and Scalable On-device Large Language Model Inference,” 2023.
- [17] J. Wei et al., “Ghidorah: Fast LLM Inference on Edge with Speculative Decoding and Hetero-Core Parallelism,” 2025.

- [18] K. Alizadeh et al., “LLM in a Flash: Efficient Large Language Model Inference with Limited Memory,” 2024.
- [19] ggml-org, “OpenCL backend documentation,” llama.cpp documentation, GitHub, accessed Apr. 30, 2026. [Online]. Available: <https://github.com/ggml-org/llama.cpp/blob/master/docs/backend/OPENCL.md>
- [20] V. J. Reddi et al., “MLPerf Mobile Inference Benchmark: An Industry-Standard Open-Source Machine Learning Benchmark for On-Device AI,” in Proceedings of MLSys, 2022.
- [21] Z. Li, M. Paolieri, and L. Golubchik, “A Benchmark for ML Inference Latency on Mobile Devices,” in Proceedings of EdgeSys, 2024.
- [22] Khronos Group, “OpenCL — The Open Standard for Parallel Programming of Heterogeneous Systems,” Khronos, accessed May 4, 2026. [Online]. Available: <https://www.khronos.org/opencl/>
- [23] NVIDIA, “CUDA Programming Guide,” NVIDIA Documentation, accessed May 4, 2026. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-programming-guide/>
- [24] Android Developers, “Android Debug Bridge (adb),” Android Developers documentation, accessed May 4, 2026. [Online]. Available: <https://developer.android.com/tools/adb>
- [25] ggml-org, “LLaMA.cpp HTTP Server,” llama.cpp documentation, GitHub, accessed May 4, 2026. [Online]. Available: <https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md>
- [26] ggml-org, “llama.cpp: LLM inference in C/C++,” GitHub repository documentation, accessed May 4, 2026. [Online]. Available: <https://github.com/ggml-org/llama.cpp>

Appendices

Appendix A: Prompt-fixture suite summary

The final workload uses a fixed synthetic prompt-fixture suite grouped by prompt-token length. The suite is versioned as 1.0.1 and is stored in `configs/prompts/dataset_suite_v1.json`. It is not a semantic answer-quality benchmark. Its purpose is to define controlled prompt-length workloads for measuring laptop-boundary TTFT, decode TPS, latency behavior, stability, and reproducibility.

Table Appendix.1: Prompt suite details

Tier	Prompt IDs	Number of prompts	Fixture token range	Role in the study
Short	final_short_01 to final_short_05	5	107–110	Light interactive workload with lower prefill pressure
Medium	final_medium_01 to final_medium_05	5	480–494	Moderate prompt-processing workload
Long	final_long_01 to final_long_05	5	1372–1400	Prefill-heavy workload used to stress first-token latency
Soak	final_soak_01	1	487	Fixed repeated prompt for sustained-performance runs

Each prompt fixture includes a prompt ID, tier, fixture token count, prompt hash, tokenizer-runtime metadata, and source/offline fixture provenance. Smoke and development prompts are not final dissertation evidence.

Appendix B: Condition matrix and regimes

The benchmark conditions compare one practical laptop CPU-only baseline with two reverse-tethered phone conditions. The condition matrix is stored in `configs/conditions/matrix.json`.

Table Appendix.2: Benchmark condition matrix

Condition ID	Device / path	Execution path	Transport	Purpose
yoga_cpu_local_Q4_0	Lenovo Yoga laptop	Local CPU llama.cpp server	Local endpoint	Practical CPU-only legacy-laptop baseline
s25ultra_cpu_phone_Q4_0	Samsung Galaxy S25 Ultra	Phone CPU llama.cpp server	USB/ADB forwarded endpoint	Diagnostic reverse-tethered phone condition
s25ultra_openc1_phon_e_Q4_0	Samsung Galaxy S25 Ultra	Phone OpenCL/GPU llama.cpp server	USB/ADB forwarded endpoint	Main implemented phone acceleration condition

Table Appendix.3: Benchmark regime matrix

Regime	Definition	Prompt coverage	Final records	Interpretation
Warm	Repeated operation after the server/model is already available	Short, medium, long	225	Primary evidence for normal repeated interactive use
Cold	First measured requests after fresh server launch/readiness	One prompt per short/medium/long tier, repeated	45	Secondary evidence for restart-adjacent behavior
Soak	Repeated fixed-prompt workload	Soak prompt only	205	Sustained-performance trend evidence, not causal thermal proof

Bundled cold-run directories are not counted as additional independent evidence beyond the final aggregation policy.

Appendix C: Benchmark commands and workflow notes

The following commands summarize the benchmark workflow. They are representative of the implemented command-line procedure and should be interpreted together with the final metadata files in results/final_input/.

Phone transport setup on the laptop:

Bash

```
./scripts/adb_bootstrap.sh
python -m client.preflight
```

Local laptop endpoint preflight:

Bash

```
python -m client.preflight --skip-adb
```

Laptop CPU server launch pattern:

Bash

```
llama-server.exe ^
-m C:\workspace\TUD\llama.cpp_windows\llama-3.2-1b-instruct-q4_0.gguf ^
-c 2048 ^
--host 127.0.0.1 ^
--port 8080 ^
-ngl 0 ^
--cache-ram 0 ^
-sps 0.0 ^
-np 1
```

Phone server launch pattern on the phone:

```
Termux
LD_LIBRARY_PATH=/vendor/lib64 ./build/bin/llama-server \
-m models/llama-3.2-1b-instruct-q4_0.gguf \
-c 2048 \
--host 127.0.0.1 \
--port 8080 \
-ngl 0 \ (# -ngl 99 for OpenCL/GPU)
--cache-ram 0 \
-sps 0.0 \
-np 1
```

Representative laptop-baseline benchmark-harness command, run after local server launch:

```
Bash
python -m client.matrix \
--node yoga \
--backend cpu \
--regimes warm \
--repetitions 5 \
--all-final-prompts \
--server-mode local \
--cache-policy disabled \
--model-name unnicknameable/Llama-3.2-1B-Instruct-Q4_0-GGUF \
--model-filename llama-3.2-1b-instruct-q4_0.gguf \
--model-sha256 a0920200b067d24e88c38688a9306de314ef1f7582eaa767343ddab89a83fa81 \
--parameter-count 1B \
--llama-cpp-commit f93c09e2673090b59802f2552848a1cac70598b0
```

The Yoga CPU server arguments are verified from final run metadata. The executable path may vary by local installation, but the benchmarked server arguments were: model path, context length 2048, host 127.0.0.1, port 8080, -ngl 0, --cache-ram 0, -sps 0.0, and -np 1.

Representative benchmark-harness command for the phone baseline:

```
Termux
python -m client.matrix \
--node s25ultra \
--backend cpu \ (# --backend opengl for phone OpenCL/GPU)
--regimes warm \
--repetitions 5 \
--all-final-prompts \
--server-mode phone \
--cache-policy disabled \
--model-name unnicknameable/Llama-3.2-1B-Instruct-Q4_0-GGUF \
--model-filename llama-3.2-1b-instruct-q4_0.gguf \
--model-sha256 a0920200b067d24e88c38688a9306de314ef1f7582eaa767343ddab89a83fa81 \
--parameter-count 1B \
--llama-cpp-commit 13d36cf89178354d9aa6732e5930d89d64caf718
```

Aggregation commands:

Bash

```
python analysis/aggregate.py \  
  --input results/final_input \  
  --output results/final_summaries_all_regimes
```

Bash

```
python analysis/aggregate.py \  
  --input results/final_input \  
  --output results/final_summaries_all_regimes_by_prompt \  
  --group-by-prompt-id
```

Cold runs required server restart/readiness between measured requests. Soak runs used the fixed soak prompt and repeated it to produce sustained-run timing records. Full reproduction should use the metadata.json files in each final input directory to confirm exact server launch arguments, commits, and model hashes.

Appendix D: Codebase structure

The repository is organized as a benchmark framework rather than as a consumer application. The following map identifies the main directories and their roles.

Table Appendix.4: Codebase roles

Directory / file group	Role	Important examples
client/	Laptop-side benchmark client, CLI, matrix runner, streaming parser, metric logic, and preflight checks	client/cli.py, client/benchmark.py, client/matrix.py, client/metrics.py, client/preflight.py
configs/prompts/	Prompt fixtures used by smoke tests and final evidence	dataset_suite_v1.json, smoke_suite.json
configs/conditions/	Regime and condition matrix definitions	matrix.json
docs/	Operational documentation and runbooks	phone_server_runbook.md, gpu_opencl_validation.md, logging_schema.md, steady_state_and_thermals.md
scripts/	ADB setup, server launch, metadata capture, cold-run bundling, and token-count verification helpers	adb_bootstrap.sh, launch_phone_server.sh, capture_phone_metadata.sh, bundle_cold_runs.py, verify_token_counts.py
analysis/	Aggregation, filtering, summary generation, plotting, and paper/report asset export	aggregate.py, export_paper_assets.py
tests/	Automated validation for metrics, streaming parser behaviour, CLI/config handling, preflight checks, aggregation, and mock end-to-end pipeline behaviour	test_metrics.py, test_benchmark.py, test_cli.py, test_preflight.py, test_aggregate.py, test_e2e_pipeline.py
results/final_input/	Raw final benchmark input directories with raw_metrics.jsonl and metadata.json	Yoga CPU, S25 CPU, and S25 OpenCL/GPU warm/cold/soak directories

Directory / file group	Role	Important examples
results/final_summaries_all_regimes/	Authoritative condition/regime/tier summary outputs	aggregation_20260426_232956

This appendix is intended as a navigation aid. The dissertation does not treat repository files as academic references; they are project artifacts that support reproducibility.

Appendix E: Representative JSONL record categories

Each benchmark run writes one JSON object to raw_metrics.jsonl. The schema is designed to preserve condition identity, prompt identity, model/runtime metadata, timing fields, computed metrics, cache-policy metadata, and failure information.

Table Appendix.5: JSON Fields and purposes

Field category	Example fields	Purpose
Run identity	timestamp, run_id, regime, repetition_index	Identifies a measured request
Condition identity	benchmark_condition_id, node, backend, server_mode	Links the record to Yoga CPU, S25 CPU, or S25 OpenCL/GPU condition
Prompt metadata	prompt_suite_id, prompt_suite_version, prompt_suite_type, prompt_id, prompt_tier, fixture_prompt_token_count	Ensures prompt fixture traceability
Model/settings metadata	model_name, model_filename, model_sha256, parameter_count, quantization, context_length, seed, temperature, max_new_tokens	Confirms comparable model and generation settings
Runtime metadata	llama_cpp_commit, llama_cpp_build_flags, server_launch_args	Documents reproducibility variables
Timing fields	request_sent_timestamp, first_token_timestamp, final_token_timestamp	Supports TTFT and decode-window calculation
Computed metrics	ttft_ms, decode_tps, generated_token_count	Provides raw metrics for aggregation
Cache-policy fields	cache_policy, cache_expected, cache_observed, cache_mismatch	Prevents hidden prompt-cache reuse from entering final evidence
Failure fields	stop_reason, known_anomalies, notes	Supports failure counts and auditability
Optional environment fields	start_temperature_c, end_temperature_c, temperature_source, battery_status	Context only; missing values are not inferred

Representative compact record, shortened for readability:

JSON

```
{
  "timestamp": "2026-04-26T19:14:23.128820",
  "regime": "warm",
  "repetition_index": 0,
  "benchmark_condition_id": "s25ultra_openc1_phone_Q4_0",
  "node": "s25ultra",
  "backend": "openc1",
  "server_mode": "phone",
  "prompt_suite_id": "dataset_suite_v1",
  "prompt_suite_version": "1.0.1",
  "prompt_suite_type": "synthetic",
```

JSON

```
"prompt_id": "final_long_01",
"prompt_tier": "long",
"fixture_prompt_token_count": 1373,
"runtime_prompt_eval_token_count": 1374,
"model_filename": "llama-3.2-1b-instruct-q4_0.gguf",
"model_sha256": "a0920200b067d24e88c38688a9306de314ef1f7582eaa767343ddab89a83fa81",
"llama_cpp_commit": "13d36cf89178354d9aa6732e5930d89d64caf718",
"quantization": "Q4_0",
"context_length": 2048,
"seed": 42,
"temperature": 0.0,
"max_new_tokens": 512,
"generated_token_count": 512,
"stop_reason": "limit",
"ttft_ms": 2566.933,
"decode_tps": 30.516,
"cache_policy": "disabled",
"cache_expected": false,
"cache_observed": "full_eval",
"cache_mismatch": false,
"mock_mode": false,
"start_temperature_c": null,
"end_temperature_c": null,
"temperature_source": ""
}
```

The example shows why thermal results are treated cautiously: timing and cache metadata are present, but usable temperature telemetry is not.

Appendix F: Result provenance

This appendix records result provenance for the empirical evidence used in Chapter 7. The dissertation uses the final aggregation outputs rather than manually interpreting raw folders. The primary grouped summary is results/final_summaries_all_regimes/aggregation_20260426_232956/. The secondary by-prompt summary is results/final_summaries_all_regimes_by_prompt/aggregation_20260426_232959/. Raw final input records are stored under results/final_input/. The tables below identify the authoritative summary files, raw input directories, final record counts, and failure counts used by the dissertation.

Table Appendix.6: Artifact roles

Artifact	Role
results/final_summaries_all_regimes/aggregation_20260426_232956/summary.csv	Primary grouped summary by condition, regime, and prompt tier
results/final_summaries_all_regimes/aggregation_20260426_232956/summary.md	Human-readable version of the primary summary
results/final_summaries_all_regimes_by_prompt/aggregation_20260426_232959/summary.csv	Secondary prompt-level summary

Artifact	Role
results/final_summaries_all_regimes_by_prompt/aggregation_20260426_232959/summary.md	Human-readable prompt-level summary

Table Appendix.7: Raw input directories

Regime	Condition	Raw input directory	Final records
Warm	Yoga CPU	results/final_input/20260426_172445_yoga_cpu_matrix	75
Warm	S25 CPU	results/final_input/20260426_174836_s25ultra_cpu_matrix	75
Warm	S25 OpenCL/GPU	results/final_input/20260426_191422_s25ultra_openc1_matrix	75
Cold	Yoga CPU	results/final_input/20260426_195648_yoga_cpu_cold	15
Cold	S25 CPU	results/final_input/20260426_200435_s25ultra_cpu_cold	15
Cold	S25 OpenCL/GPU	results/final_input/20260426_201450_s25ultra_openc1_cold	15
Soak	Yoga CPU	results/final_input/20260426_202558_yoga_cpu_soak	60
Soak	S25 CPU	results/final_input/20260426_213408_s25ultra_cpu_soak	70
Soak	S25 OpenCL/GPU	results/final_input/20260426_222123_s25ultra_openc1_soak	75

Table Appendix.8: Record totals

Regime	Final records	Failure count
Warm	225	0
Cold	45	0
Soak	205	0

Soak elapsed-time files record approximately 21.38 minutes for Yoga CPU, 18.36 minutes for S25 CPU, and 25.39 minutes for S25 OpenCL/GPU. The phone thermal_adb.csv files contain only headers, and raw JSONL temperature fields are unavailable, null, or zero. Therefore, soak figures are sustained-performance trend evidence only and are not used as proof of causal thermal throttling.

Bundled cold-run directories are not counted as additional independent evidence beyond the final aggregation policy.

Appendix G: Test evidence

Automated testing was verified using Python unittest discovery from the repository root.

Bash
PYTHONPATH=. python -m unittest discover -s tests -v

Result: 152 tests run, 0 failures, and 0 errors.

The command python -m pytest -q was attempted but was not completed within the tool timeout, so the dissertation does not claim a passing pytest run.

Table Appendix.9: Representative automated test coverage

Test area	Evidence / files	Purpose
Metric calculation	tests/test_metrics.py, client/metrics.py	Verifies TTFT and decode TPS calculations, including zero/one-token and invalid-window cases
Streaming parser and first-token detection	tests/test_benchmark.py, client/benchmark.py	Verifies SSE parsing, non-empty first-token detection, final-event handling, and token-count extraction
Prompt/config handling	tests/test_cli.py, configs/prompts/	Verifies prompt suite shape, prompt tiers, final metadata, and smoke-suite separation
Matrix runner	tests/test_benchmark.py, client/matrix.py	Verifies repetition logic, regime labels, prompt ordering, and failed-run logging
Preflight checks	tests/test_preflight.py, client/preflight.py	Verifies ADB, port-forwarding, server-health, timeout, and connection-refused cases
Aggregation and final-evidence filtering	tests/test_aggregate.py, analysis/aggregate.py	Verifies strict final-evidence filtering, p50/p95/mean outputs, failure counts, and by-prompt grouping
Mock end-to-end pipeline	tests/test_e2e_pipeline.py	Verifies pipeline behavior without relying on external device state

These tests support benchmark-framework correctness. They do not establish production reliability, long-term robustness, or correctness of all possible Android/ADB/OpenCL runtime states.

Appendix H: Technical-user evaluation template

This appendix records the technical-user evaluation instrument prepared for a reproducibility and usability review of the benchmark framework. It is not a literature-review comprehension checklist. The evaluation asks whether a technically competent peer can understand the project purpose, prepare the environment, validate installation, inspect prompts and conditions, identify the local baseline workflow, identify the phone CPU/OpenCL workflow if device access exists, run or follow a small benchmark command, locate raw_metrics.jsonl, run aggregation, locate summary.csv or summary.md, interpret TTFT and decode TPS, understand non-goals, and report unclear documentation or setup steps. Completed peer feedback was unavailable at the time of writing; therefore no peer outcomes are claimed.

Table Appendix.10: Technical-user evaluation template

Evaluation stage	Task / question	Expected evidence	Response type
Background and role	What is your technical background, and how familiar are you with Python, command-line tools, Android/ADB, and LLM inference?	Evaluator states relevant experience level.	Short comment
Project purpose	Could you explain the purpose of the reverse-tethered benchmark framework in one or two sentences?	Evaluator identifies that the laptop remains the client while the phone is used as a local inference node.	Yes / Partially / No + comment
Environment preparation	Could you identify the required Python environment, model file, configuration files, and device/server prerequisites from the documentation?	Evaluator can list the required setup elements or identify missing/ambiguous steps.	Yes / Partially / No + comment

Evaluation stage	Task / question	Expected evidence	Response type
Installation validation	Could you run the preflight or validation step and understand whether the benchmark environment was ready?	Successful preflight output or clear explanation of the failure.	Yes / Partially / No + comment
Configuration inspection	Could you locate the prompt suite and condition matrix used by the benchmark?	Evaluator identifies configs/prompts/dataset_suite_v1.json and configs/conditions/matrix.json.	Yes / Partially / No + comment
Local baseline workflow	Could you identify how the local Yoga CPU baseline server and benchmark-harness workflow are started?	Evaluator can point to the local server launch pattern and benchmark-harness command.	Yes / Partially / No + comment
Phone workflow	If device access exists, could you identify the ADB setup and phone CPU/OpenCL server workflow?	Evaluator can identify ADB forwarding, phone server launch mode, and preflight validation.	Yes / Partially / No + comment
Benchmark execution	Could you run a short validation benchmark or follow the benchmark command structure without manual intervention from the author?	raw_metrics.jsonl or equivalent output artifact generated, or a clear blocker documented.	Yes / Partially / No + comment
Raw log discovery	Could you locate raw_metrics.jsonl and understand what it records?	Evaluator identifies the raw record file and its main timing/metadata fields.	Yes / Partially / No + comment
Aggregation	Could you run the aggregation step and locate summary.csv or summary.md?	Aggregated summary artifact found or aggregation failure clearly explained.	Yes / Partially / No + comment
Result interpretation	Could you explain TTFT and decode TPS from the generated summary?	Evaluator correctly distinguishes first-token responsiveness from decode-phase throughput.	Yes / Partially / No + comment
Failure/debug clarity	If a server, ADB, or preflight step fails, is the troubleshooting path clear?	Evaluator identifies the likely failure source and the next diagnostic step.	Yes / Partially / No + comment
Scope-boundary understanding	Was it clear that the framework does not implement NPU acceleration, semantic answer-quality scoring, or production chatbot UX?	Evaluator can state the non-goals correctly.	Yes / Partially / No + comment
Documentation clarity	Which setup, command, or interpretation step was least clear?	Specific unclear step identified.	Short comment
Open feedback	What one change would most improve reproducibility, usability, or confidence in the benchmark framework?	Actionable suggestion recorded.	Short comment

Status: template prepared; feedback pending or unavailable at the time of writing; no peer-evaluation outcomes are claimed.